SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1004	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	AKSHAYA SEKAR	**Reg. No.:**	1911260204
Date:	29/10/26	**Duration:**	60 minutes

Code of Conduct

I AKSHAYA SEKAR (1911260204) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: AKSHAYA SEKAR

ABSTRACT

The Intelligent Hotel Management and Guest Experience System is proposed to integrate room inventory, reservations, check-in and check-out, housekeeping, guest requests, billing, guest histories, and hotel performance analytics. The problem statement notes that fragmented systems can cause booking conflicts and delays in responding to guests. A shared system can improve the consistency and visibility of hotel operations.

This report recommends a layered architecture with modular business components. Guest and staff interfaces communicate through an application/API layer with the reservation, guest-stay, housekeeping, guest-service, billing, and analytics modules. A relational database stores core operational records and supports transactional updates. Optional external integrations can provide payment processing and email or SMS notifications.

The design prioritizes data integrity, security, performance, reliability, availability, scalability, and maintainability. Reservation and allocation of workflows must prevent conflicting room assignments; housekeeping must distinguish vacant rooms from rooms that are ready; and billing must retain traceable charges. The architecture is a logical proposal whose detailed targets and implementation choices should be validated with hotel stakeholders.

REPORT ORGANIZATION

- 1. Introduction and objectives
- 2. Requirements analysis
- 3. Architecture selection
- 4. Architecture diagram and components
- 5. Workflow and data management
- 6. Quality attributes
- 7. Design decisions and future scope
- 8. Testing, risks, and conclusion

1. INTRODUCTION AND SYSTEM OBJECTIVES

1.1 Problem Overview

Hotels coordinate several connected activities. A reservation affects room availability; check-in creates an active stay; housekeeping determines whether a room is ready; guest requests require assignment and follow-up; and check-out requires accurate billing. If these activities use disconnected records, staff may see outdated information, assign an unavailable room, miss a request, or produce an incorrect invoice.

The proposed system creates a shared operational view so authorized users can work with consistent room, reservation, guest, stay, task, request, and billing information. The objective is to reduce duplication and make the status of each workflow visible.

1.2 Objectives

- Manage room inventory, room types, room status, and availability.
- Process reservations and validate room availability before confirmation.
- Manage guest check-in, room allocation, active stays, and check-out.
- Track housekeeping tasks and confirm room readiness.
- Record, assign, and monitor guest service requests.
- Generate itemized bills and invoices from authorized charges.
- Maintain permitted guest history for reservations, stays, requests, and invoices.
- Generate performance analytics for occupancy, revenue, service response, and housekeeping.

1.3 Scope and Assumptions

The core scope covers the eight objectives above. Payment gateways, messaging providers, external identity systems, and integrations with other hotel platforms are optional interfaces. The problem statement does not specify hotel size, hosting environment, payment provider, tax jurisdiction, or legal retention rules; these must be confirmed before implementation.

2. SYSTEM REQUIREMENTS ANALYSIS

2.1 Functional Requirements

FR1 – Room inventory: Storeroom number, type, capacity, rate, and operational status; permit authorized updates.

FR2 – Reservations: Create, update, retrieve, and cancel reservations; validate dates, capacity, and availability.

FR3 – Check-in and allocation: Verify the booking, confirm room readiness, allocate an eligible room, and activate the stay.

FR4 – Check-out: Calculate outstanding charges, issue an invoice, close the stay, and send the room to be housekeeping.

FR5 – Housekeeping: Create and assign tasks, record progress, and update room readiness after required checks.

FR6 – Guest requests: Record request type, priority, room/stay reference, owner, status, and resolution.

FR7 – Billing: Combine room and additional charges, apply configured fees, and produce itemized invoices.

FR9 – Analytics: Report on occupancy, revenue, reservation patterns, service response, and housekeeping completion.

2.2 Non-Functional Requirements

- Security: authentication, role permissions, encryption, and audit logs.
- Performance: keep common searches and operational actions responsive under expected load.
- Reliability: transactions and validation prevent partial or conflicting updates.
- Availability: monitor the service and provide backup and recovery procedures appropriate to hotel needs.
- Scalability: support growth in rooms, reservations, users, tasks, and reporting volume.

3. PROPOSED SOFTWARE ARCHITECTURE

3.1 Selected Style

A layered architecture with modular business components and a client-server interaction model is recommended. The presentation layer provides guest and staff interfaces; the application/API layer validates requests and enforces access rules; business modules implement hotel workflows; the data-access layer manages persistence; and the analytics layer prepares reports.

This design suits a system whose functions are closely related and must share consistent data. Reservation, allocation, guest-stay, housekeeping, and billing workflows interact frequently. A modular application can keep these responsibilities distinct while allowing important operations to use reliable database transactions.

3.2 Layers and Responsibilities

Presentation layer: Guest portal, front-desk console, housekeeping dashboard, finance screens, and management dashboard.

Application/API layer: Defined endpoints, input validation, authentication, permission checks, and coordination of business operations.

Business layer: Reservation rules, guest and stay in workflows, housekeeping tasks, service requests, billing, and report coordination.

Data-access layer: Reusable queries and transaction handling, keeping direct database access out of user-interface code.

Data layer: Relational storage for rooms, reservations, guests, stays, tasks, charges, invoices, and audit records.

Analytics layer: Aggregated operational reports designed to minimize impact on time-sensitive transactions.

3.3 Architectural Fit

4. ARCHITECTURE DIAGRAM AND COMPONENTS

Figure 1 shows how guest and staff interfaces send requests through the application/API layer to the appropriate business component. Business modules use controlled data access to maintain operational records. Analytics uses suitable aggregate data. Payment and messaging providers connect through explicit interfaces when required.

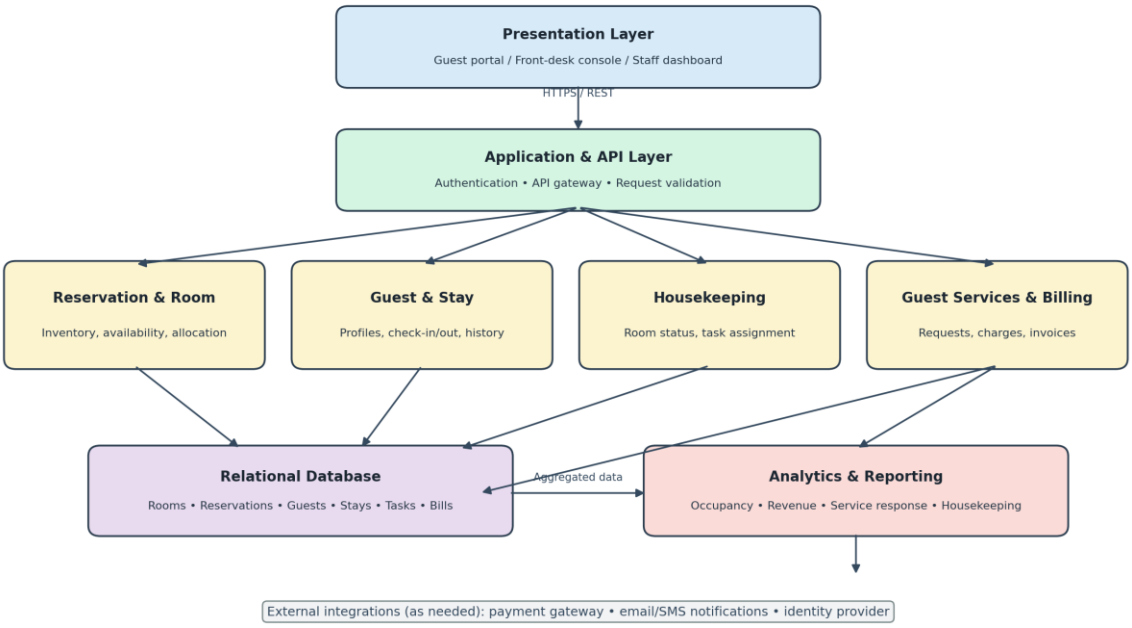


Figure 1. Proposed architecture of the Intelligent Hotel Management and Guest Experience System

4.1 Component Responsibilities

Reservation and Room: Checks inventory, availability, and allocation.

Guest and Stay: Maintains profiles and stay links.

Housekeeping: Assigns tasks and tracks room readiness.

Guest Services: Routes requests and tracks resolution.

Billing: Consolidates charges and creates invoices.

Analytics: Reports of occupancy, revenue, and service performance.

5. SYSTEM WORKFLOW AND DATA MANAGEMENT

5.1 Reservation Workflow

1. Guest or staff enter dates, guest count, and room preferences.
2. System searches inventory and checks availability, date validity, and room capacity.
3. System revalidates availability during confirmation and stores the reservation with a unique reference and status.

4. Availability views are updated; a configured service may send confirmation.

Simultaneous booking attempts require concurrency control. Availability must be rechecked inside a transaction or equivalent mechanism at confirmation time. If the room is no longer available, the system should decline that allocation and offer alternatives.

5.2 Check-in and Check-out

At check-in, staff retrieve the reservation, verify required guest details, confirm room readiness, allocate the room, and activate a stay record. During the stay, permitted charges and guest requests are linked to that stay. At check-out, the system calculates room and additional charges, creates an itemized invoice, records payment status, and closes the stay. The room then enters housekeeping and is not treated as ready until required checks are complete.

5.3 Housekeeping and Guest Requests

Housekeeping tasks include a room for reference, task type, priority, assignee, and status. Staff update progress and completion; inspection rules determine when the room becomes ready. Guest requests include category, description, priority, room/stay reference, responsible department, timestamps, and resolution. Clear ownership and statuses help prevent requests from being overlooked.

5.4 Billing and Analytics

6. QUALITY ATTRIBUTE ANALYSIS

Scalability:

Modular components, indexed queries, pagination, and horizontal application scaling can support growth. Availability must still be revalidated before confirmation. Reporting aggregation or a separate reporting store can be added as volumes increase.

Security:

Use secure authentication, role-based permissions, least privilege, HTTPS, suitable password hashing, protected secrets, and audit logs. Guests should access only their own permitted records. Use a compliant payment provider where possible instead of storing card details directly.

Performance:

Optimize availability queries and indexes, avoid repeated calls, paginate large lists, and separate heavy reports from front-desk transactions. Response-time targets must be agreed and load-tested.

Reliability:

Use database transactions, validation, unique identifiers, backups, monitoring, and tested recovery. External payment or messaging failures must not corrupt core records; pending or failed states and safe retries should be supported.

Maintainability:

Use documented API contracts, clear module ownership, code reviews, version control, and unit/integration tests. Centralize cancellation, pricing, tax, room-readiness, and invoice-adjustment rules.

Availability:

Use health monitoring, recovery procedures, controlled deployments, and tested backups. Redundant application instances and database replication can be considered if required service levels to justify their cost.

6.1 Quality Evaluation

7. ARCHITECTURAL DECISIONS AND FUTURE SCOPE

7.1 Design Decisions

Layered structure: Separates interface, application coordination, business rules, and persistence for clearer testing and maintenance.

Modular business capabilities: Keeps reservations, stays, housekeeping, guest requests, billing, and analytics logically distinct.

Relational database: Supports structured relationships, transactions, and constraints for bookings, room allocation, and invoices.

Central authorization: Provides consistent permission checks across different interfaces.

Explicit statuses: Makes reservation, room, stay, task, request, invoice, and payment progress visible and auditable.

Separated analytics: Reduces the impact of historical reporting on operational transactions.

7.2 Trade-offs and Limitations

A shared relational database simplifies consistency but needs indexing and capacity planning. A modular single deployment is easier to operate than many microservices, though a deployment problem may affect multiple modules. Extracting services later enables independent scaling but adds network, monitoring, and distributed-failure complexity.

The benefits of modularity depend on clear boundaries. Modules should not bypass one another's rules through uncontrolled database updates. Hosting, backup frequency, privacy obligations, tax handling, and payment integration need further stakeholder confirmation.

7.3 Future Enhancements

- Responsive guest portal or mobile application for self-service requests and booking information.
- Notifications for booking confirmation, request updates, and invoice availability.
- Forecasting and pricing recommendations based on occupancy trends.
- Intelligent request routing and housekeeping scheduling based on priority and workload.

8. TESTING, RISKS, AND CONCLUSION

8.1 Testing Strategy

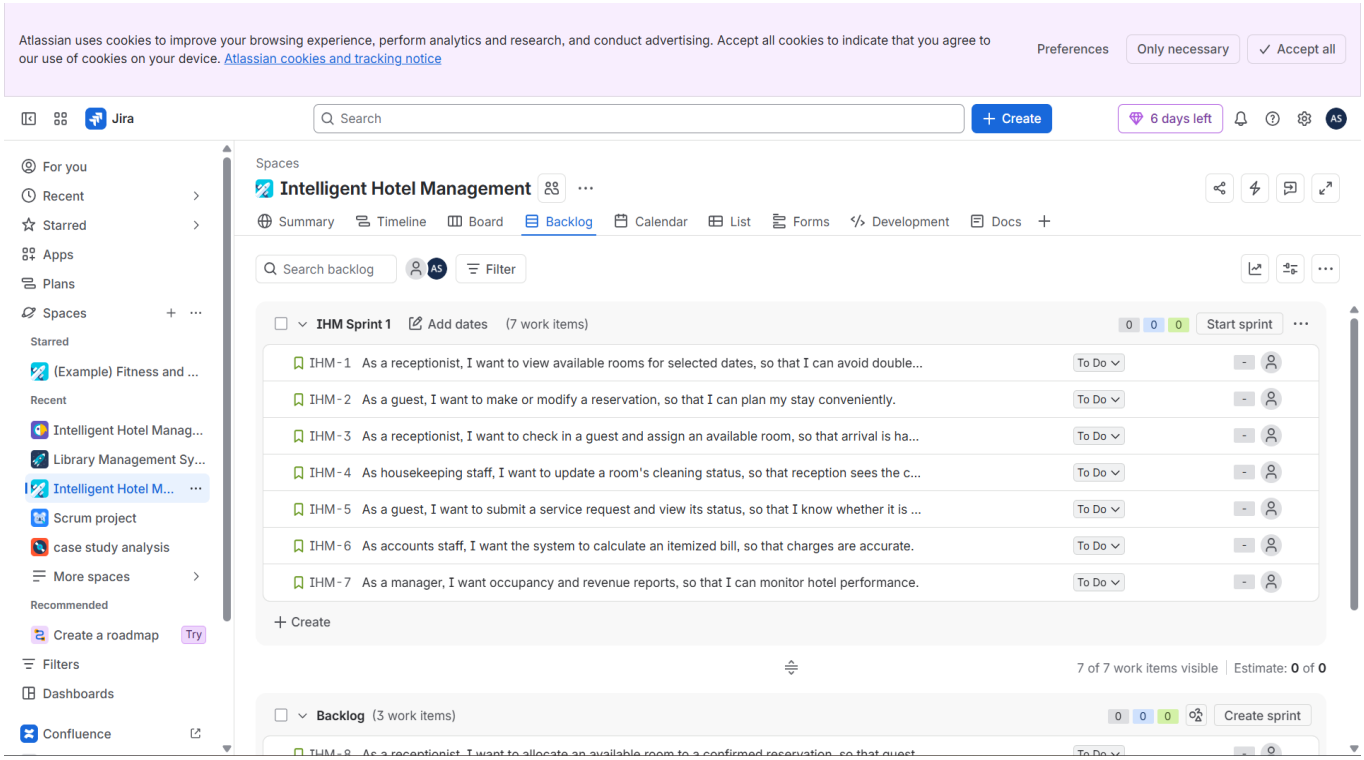
- Reservation conflict: concurrent overlapping requests must not confirm more bookings than allowed.
- Room readiness: a room under cleaning must not be treated as ready for allocation.
- Billing accuracy: expected room and additional charges must appear correctly on the invoice.
- Authorization: guests must not view other guests' records; staff access must match roles.
- Integration failure: external-service failure must not corrupt core records; retry behavior must be safe.
- Performance and recovery: test peak operational load and verify backup restoration.

8.2 Risks and Mitigation

8.3 Expected Outcomes

8.4 Conclusion

The layered architecture with modular business components integrates the hotel's core workflows and provides a maintainable foundation. Success depends on clear requirements, secure access, consistent transactions, useful reporting, testing, and staff adoption. The design can evolve as the hotel's needs grow.


Atlassian uses cookies to improve your browsing experience, perform analytics and research, and conduct advertising. Accept all cookies to indicate that you agree to our use of cookies on your device. Atlassian cookies and tracking notice
Preferences
Only necessary
Accept all
Jira
Search
Create
6 days left
For you
Recent
Starred
Apps
Plans
Spaces
Starred
(Example) Fitness and ...
Recent
Intelligent Hotel Manag...
Library Management Sy...
Intelligent Hotel M...
Scrum project
case study analysis
More spaces
Recommended
Create a roadmap
Try
Filters
Dashboards
Confluence
Spaces
Intelligent Hotel Management
Summary
Timeline
Board
Backlog
Calendar
List
Forms
Development
Docs
Search backlog
Filter
IHM Sprint 1
Add dates
(7 work items)
Start sprint
IHM-1 As a receptionist, I want to view available rooms for selected dates, so that I can avoid double...
IHM-2 As a guest, I want to make or modify a reservation, so that I can plan my stay conveniently.
IHM-3 As a receptionist, I want to check in a guest and assign an available room, so that arrival is ha...
IHM-4 As housekeeping staff, I want to update a room's cleaning status, so that reception sees the c...
IHM-5 As a guest, I want to submit a service request and view its status, so that I know whether it is ...
IHM-6 As accounts staff, I want the system to calculate an itemized bill, so that charges are accurate.
IHM-7 As a manager, I want occupancy and revenue reports, so that I can monitor hotel performance.
To Do
Create
7 of 7 work items visible
Estimate: 0 of 0
Backlog (3 work items)
Create sprint

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components,	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient	Diagram is incomplete, unclear, or technically incorrect.
	interfaces, and interactions accurately represented.		detail.	
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.

Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25